

LEO Digital Twin: Step-by-Step Process Explanation

This document provides an easily understandable, step-by-step explanation of the internal workings and processes of the I-MACSI (Intent-Aware Multi-Agent Cognitive Swarm Intelligence) LEO Satellite project.

1. Network Initialization (The Constellation)

What happens: When the application starts (via `start.bat`), the backend initializes a Digital Twin of a Low Earth Orbit (LEO) satellite network. **How it works:**

- The system generates a topology of 16 satellites distributed evenly across 4 orbital planes (4 satellites per orbit).
 - **Inter-Satellite Links (ISL):** Each satellite establishes virtual radio links with its neighbors.
 - *Intra-plane links* connect satellites in the same orbit.
 - *Inter-plane links* connect satellites in adjacent orbits.
 - The network is dynamic; it continuously tracks latency, bandwidth, and congestion for every link.
-

2. Intent Understanding Engine (Why we route)

What happens: Before a packet of data is sent, the system determines the "Intent" or purpose of the mission. **How it works:**

- A user selects a mission type (e.g., *Critical Disaster Response*, *Earth Observation*, or *Secure Mission*).
 - The **Intent Extraction Engine** converts this high-level mission into an 8-Dimensional vector: *Latency, Throughput, Reliability, Congestion, Energy, Security, Coverage, and Compute*.
 - **Why it matters:** Standard networks route everything the same way. This AI-native network routes disaster data via the *fastest* path (low latency), but routes Earth observation data via the *widest* path (high throughput), even if it takes slightly longer.
-

3. Cognitive Swarm Routing (The AI Brain)

What happens: Each satellite acts as an independent, autonomous AI agent. There is no central Earth-based controller telling them how to route traffic. **How it works (The 5-Step Loop):**

1. **Perceive:** The satellite senses the current conditions of its links (are they congested? did one fail?).
 2. **State:** It updates its internal memory.
 3. **Decide:** It consults its **Q-table** (a machine-learning lookup table) to pick the best neighbor for the packet. The decision is mathematically influenced by the Mission Intent.
 4. **Act:** It forwards the packet to the chosen neighbor (the next hop).
 5. **Learn:** It receives feedback on how successful the decision was, and updates its Q-table using the *Bellman Equation*. Over time, the satellite gets smarter and learns the optimal paths through the network.
-

4. Intent Dissemination (Swarm Communication)

What happens: Satellites share their mission intents with their neighbors. **How it works:**

- Instead of keeping the mission intent a secret, satellites use a gossip-style protocol to broadcast the intent to nearby satellites (up to 2 hops away).

- This gives the swarm "neighborhood awareness," allowing multiple satellites to proactively clear bandwidth or adjust their transmission power to help the mission succeed before the packet even arrives.
-

5. Dynamic Failure & Autonomous Recovery (Self-Healing)

What happens: Space is unpredictable. Satellites move out of range, and lasers/radios fail. The network must heal itself. **How it works:**

- If a link breaks, the satellite immediately detects that the neighbor is unreachable.
 - It records the failure locally and marks the link state as `FAILED`.
 - The AI routing algorithm (Q-Learning) immediately adjusts. The satellite automatically skips the failed neighbor and discovers an alternative path.
 - Once the link comes back online, the satellite perceives the recovery and seamlessly integrates the link back into its routing table.
-

6. The Digital Twin Dashboard (User Interaction)

What happens: The user interacts with the live simulation through a visual web interface. **How it works:**

- **Backend (Python/Flask):** Exposes APIs to calculate routes, inject link failures, and fetch network states. It also runs a WebSocket server to push real-time events.
- **Frontend (React/Vite):** Renders a 3D visualization of the Earth and the satellite swarm.
- **The Flow:**
 1. The user selects a Source, Destination, and Intent on the dashboard.
 2. The frontend sends this to the Python backend.
 3. The backend runs the AI routing simulation, discovering the optimal path.
 4. The backend streams the calculated route back via WebSockets.
 5. The frontend visualizes the packet traveling across the network as a glowing line (active route). If the user clicks a link to break it, the route glows red, and they can watch the AI instantly reroute around the failure.